XAI (Explainable AI)

> **Note**
>
> **Explainable AI (XAI)** refers to techniques that help us **understand, interpret, and trust** machine learning models by explaining *why* a model makes certain predictions.

Common XAI Techniques

LIME (Local Interpretable Model-agnostic Explanations)

- Explains **individual predictions**
- Approximates model locally with a **simple interpretable model (like linear)**
- Works for **any black-box model**

> **Use Case**
>
> Explaining why a model predicted **high pollution** at a specific time point

---

SHAP (SHapley Additive exPlanations)

- Based on **game theory**
- Assigns **feature importance values** to each input
- Provides both:
  - Local explanations (single prediction)
  - Global explanations (overall feature impact)

> **Use Case**
>
> Finding which features (e.g., temperature, wind, PM history) most influence predictions

---

Gradient-based XAI

- Uses **gradients of output w.r.t input**
- Shows **sensitivity of model output to input features**
- Common in neural networks

> **Use Case**
>
> Understanding which inputs most affect Neural ODE predictions

---

Why XAI is NOT Common in Neural ODEs

> **Warning**
>
> Neural ODEs are fundamentally different from standard ML models

- Neural ODE learns:

$$\frac{dy}{dt} = f_\theta(y)$$

- Focus is on:
  - Learning **system dynamics**
  - Modeling **continuous-time evolution**

Key Limitation

> **Danger**
>
> No clear notion of "feature importance"

- Inputs are **states evolving over time**
- Model represents a **differential equation**, not direct mapping:

```
Input → Output
```

- Instead it models:

```
State → Derivative → Evolution over time
```

So:

- XAI methods like SHAP/LIME **don't directly apply**
- Hard to interpret "which feature caused what"

My Approach (Bridging the Gap)

> **Tip**
>
> Instead of pure ODE systems, we used a **real-world noisy dataset**

In this project, we moved beyond a purely theoretical ODE system and applied our models to a **real-world noisy time-series dataset (Beijing PM2.5)**. The goal was to evaluate whether a **Neural ODE**, which is designed to learn continuous dynamics, can still capture meaningful temporal patterns in practical data. To properly benchmark its performance, we compared it against widely used models such as **Linear Regression** and **Random Forest**, which are standard choices for time-series and tabular prediction tasks.

To further understand model behavior, we incorporated **Explainable AI (XAI)** techniques. Since Neural ODEs do not naturally provide feature importance, we used XAI to interpret both traditional models and the Neural ODE indirectly.

> **Dataset: Beijing PM2.5**
>
> - Time-series pollution dataset
> - Not a true ODE system
> - Contains:
> - Environmental variables
> - Temporal patterns
> - Noise

Summary of Approach

- **Models Used**
  - Neural ODE → learn continuous-time dynamics
  - Linear Regression → simple baseline
  - Random Forest → non-linear benchmark
- **Goals**
  - Check if Neural ODE can **learn temporal structure** from noisy data
  - Compare performance with **standard ML models**
  - Evaluate generalization on real-world data
- **XAI Techniques Applied**
  - **SHAP**
    - Global + local feature importance
    - Identify key drivers of pollution (e.g., weather, past PM values)
  - **Gradient-based XAI (for Neural ODE)**
    - Measure sensitivity of outputs to inputs
    - Provide indirect insight into learned dynamics

Visualisation of Beijing PM 2.5 Dataset

> **Note**
>
> This dataset captures **air pollution trends in Beijing** along with environmental and temporal factors.

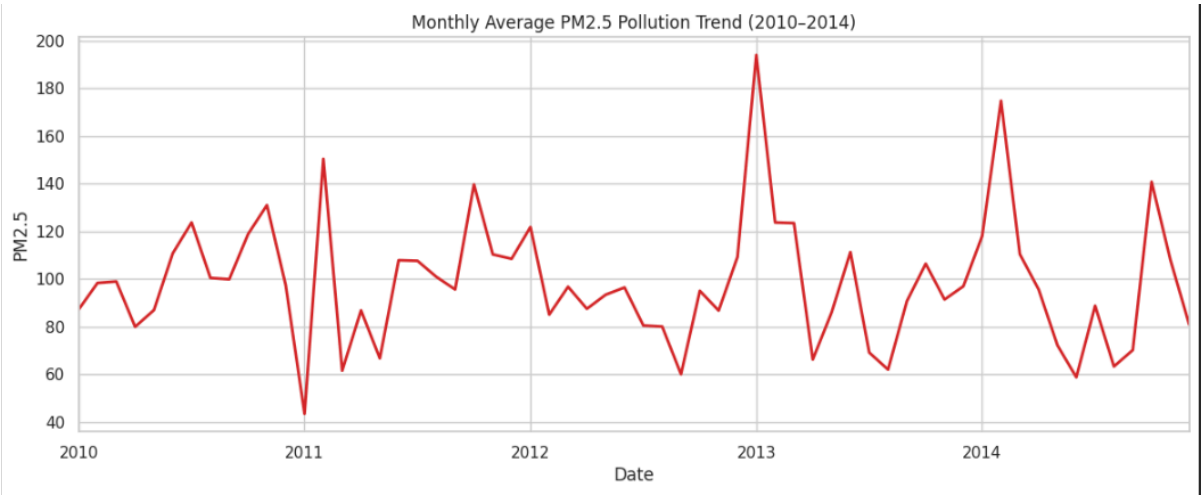
Features Overview

- **No** → Row index (not useful)
- **year, month, day, hour** → capture **temporal patterns**
- **TEMP** → affects pollution dispersion
- **DEWP** → indicates humidity
- **PRES** → high pressure traps pollution
- **cbwd** → wind direction (categorical)
- **Iws** → wind speed (reduces pollution)
- **Is, Ir** → snow/rain (clean pollutants)
- **pm2.5** → Target variable (pollution level)

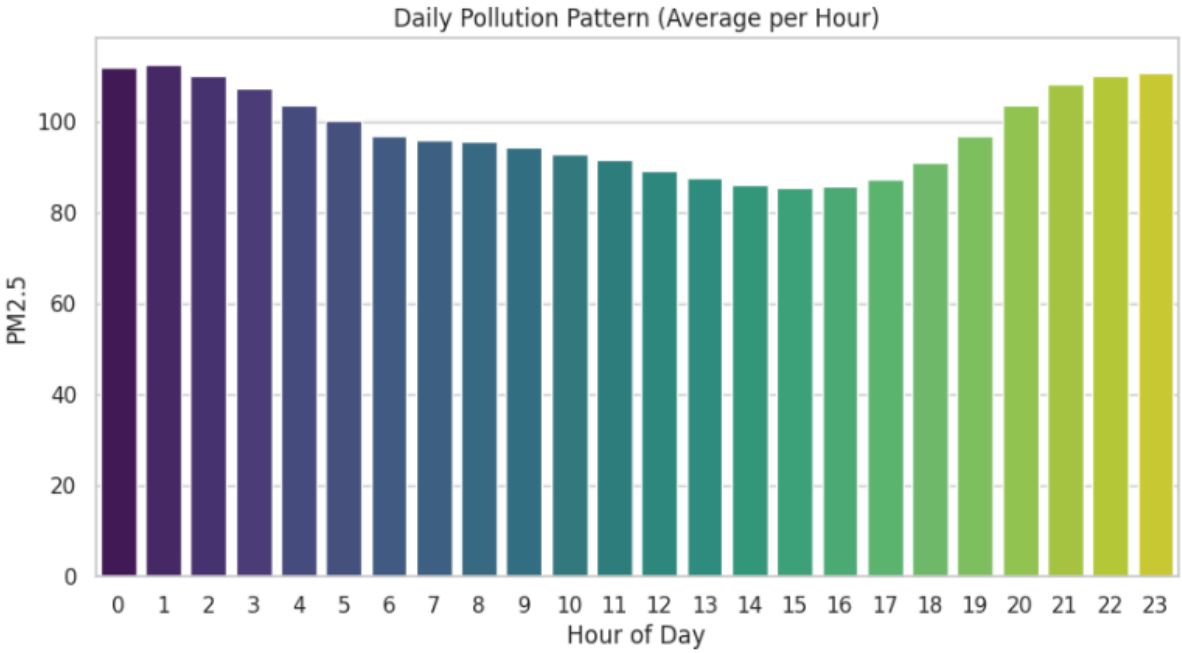
Dataset Snapshot

datetime	No	year	month	day	hour	pm2.5	DEWP	TEMP	PRES	cbwd	Iws	Is	Ir	cbwd_encoded
2010-01-02 00:00:00	25	2010	1	2	0	129.0	-16	-4.0	1020.0	SE	1.79	0	0	2
2010-01-02 01:00:00	26	2010	1	2	1	148.0	-15	-4.0	1020.0	SE	2.68	0	0	2
2010-01-02 02:00:00	27	2010	1	2	2	159.0	-11	-5.0	1021.0	SE	3.57	0	0	2
2010-01-02 03:00:00	28	2010	1	2	3	181.0	-7	-5.0	1022.0	SE	5.36	1	0	2
2010-01-02 04:00:00	29	2010	1	2	4	138.0	-7	-5.0	1022.0	SE	6.25	2	0	2

PM2.5 vs Time (2010–2014)



Daily Pollution Pattern



✓ **Observation**

- Pollution varies significantly across day

Feature Selection

Using SelectKBest + F-Regression

ⓘ **Info**

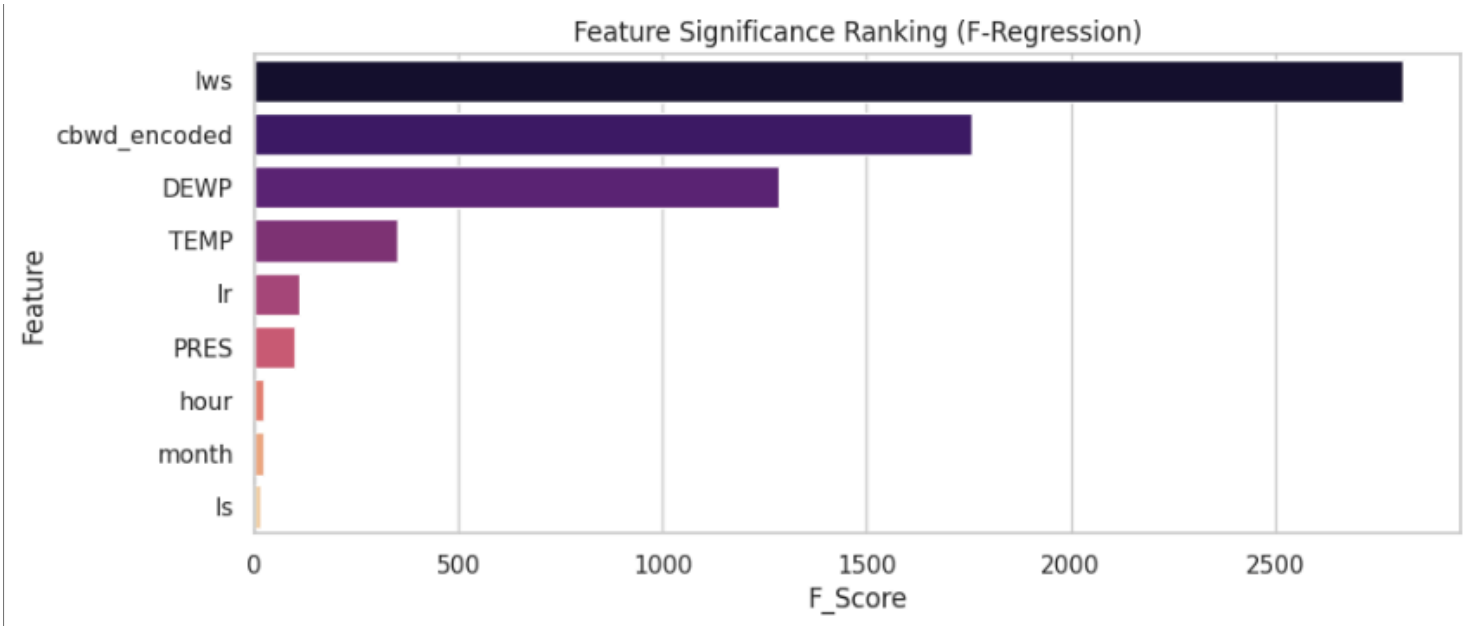
F-regression measures **linear relationship strength** between each feature and target.

- Higher F-score → more relevant feature

```
selector = SelectKBest(score_func=f_regression, k='all')
selector.fit(X_all, y)
```

- Computes **statistical significance** of each feature
- Ranks features based on predictive power

Feature Ranking



✓ **Top 6 Selected**

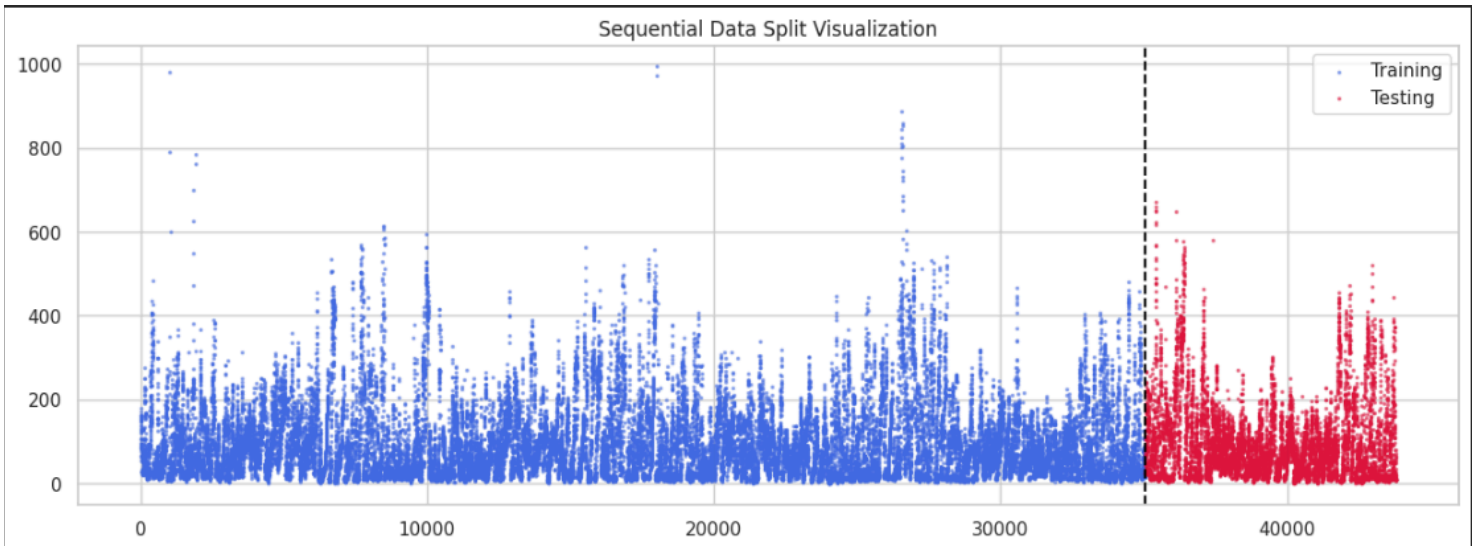
```
['Iws', 'cbwd_encoded', 'DEWP', 'TEMP', 'Ir', 'PRES']
```

Data Splitting

Example

Sequential split (time-series aware)

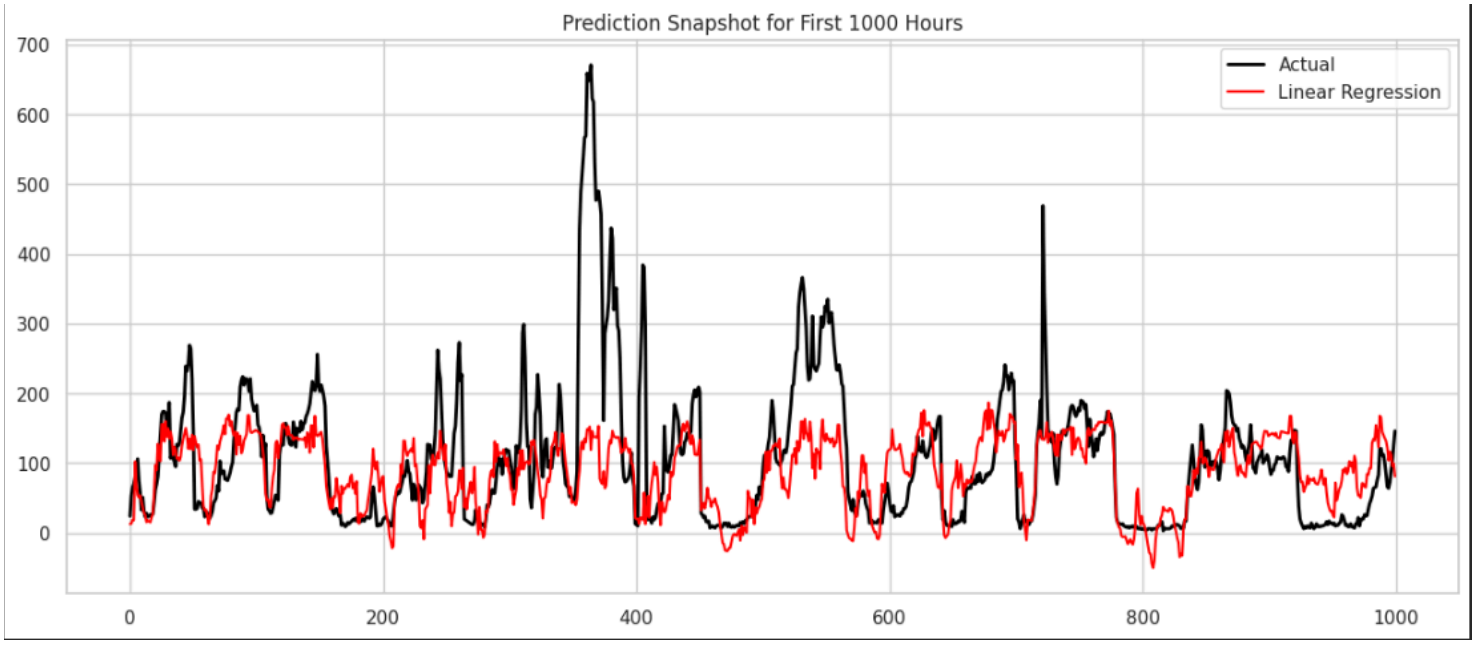
- Train: 80%
- Test: 20%



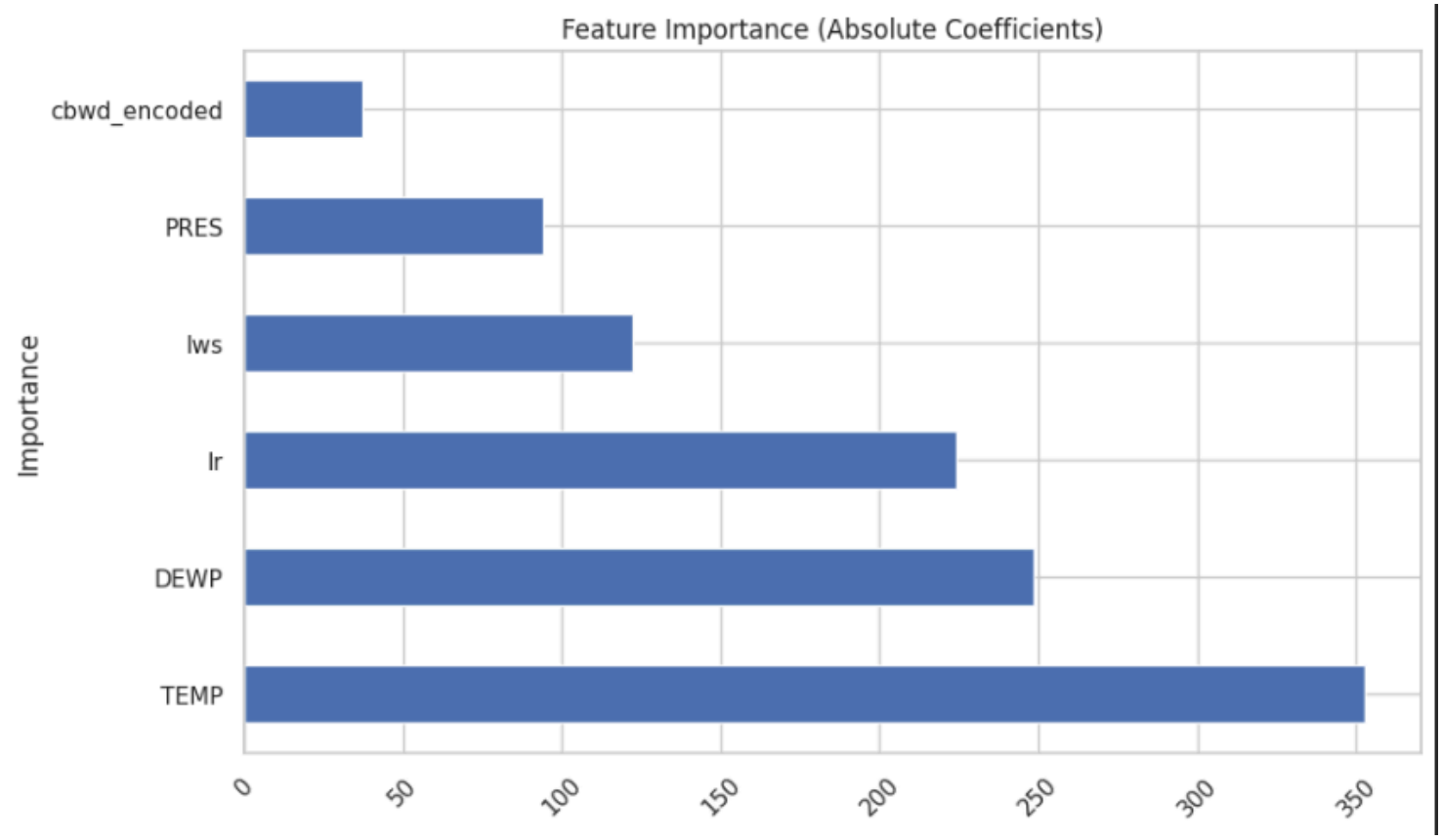
Linear Regression

✓ **Results**

- MSE: 6876
- MAE: 57.88
- R^2: 0.21



Feature Importance



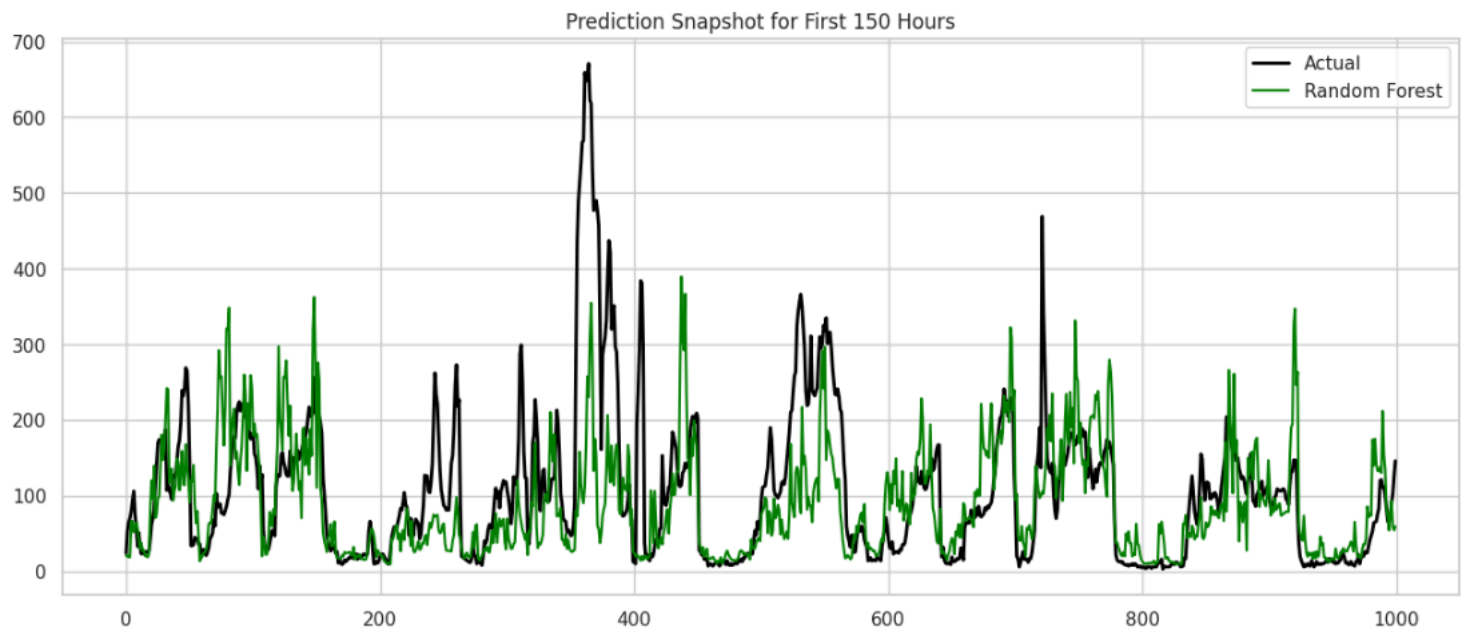
✓ **Observation**

- **TEMP, DEWP, Ir** most influential
- Linear model struggles with **non-linearity**
- Absolute coeff is not the best way to know most important features

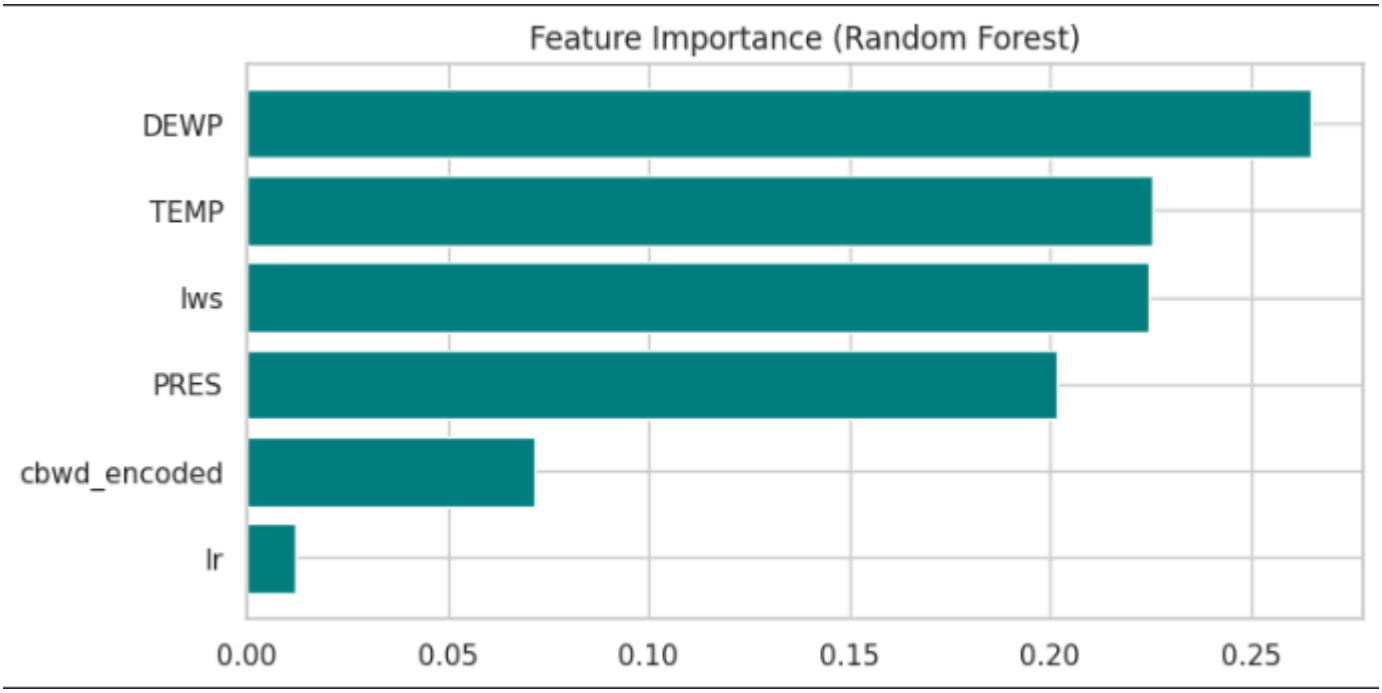
Random Forest

✓ **Results**

- MSE: 6649
- MAE: 56.23
- R^2: 0.24



Feature Importance



✓ **Observation**

- **DEWP, TEMP, Iws** dominate
- Captures non-linearity better than LR
- **Ir less important here → model difference insight**

Neural ODE

Note

Neural ODE is not ideal for tabular regression, but we test if it can **learn temporal structure**.

Sequence Creation

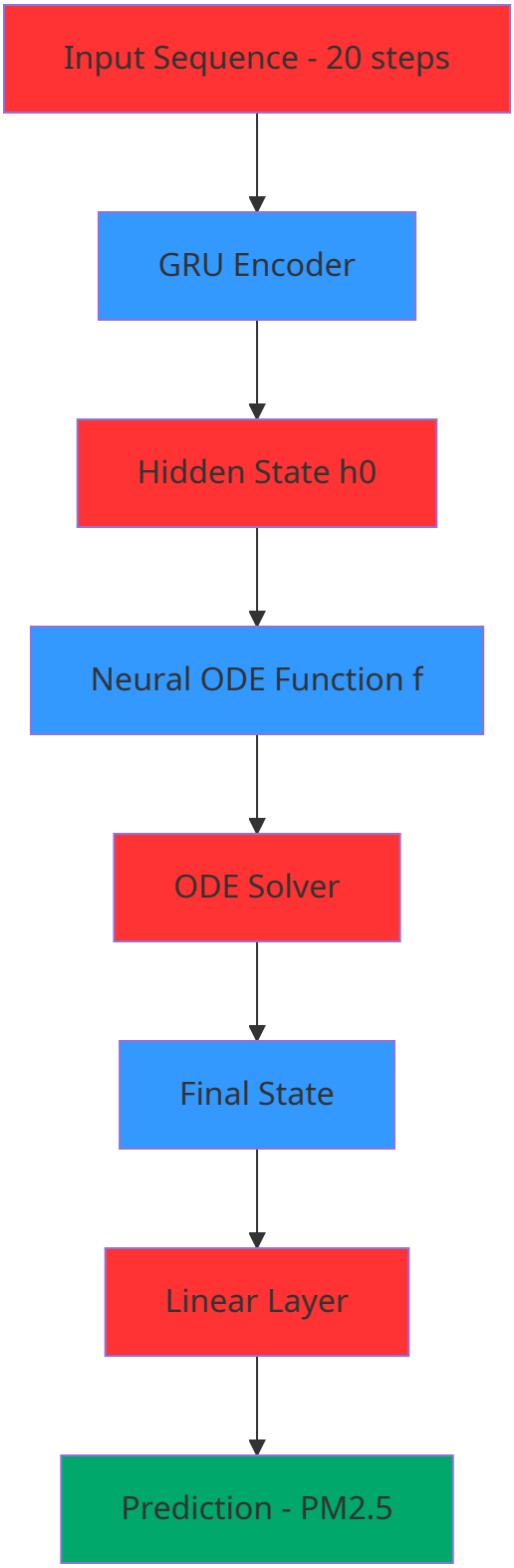
```
def create_sequences(X, y, seq_len):
    Xs, ys = [], []
    for i in range(len(X) - seq_len):
        Xs.append(X[i:i+seq_len])
        ys.append(y[i+seq_len])
    return np.array(Xs), np.array(ys)
```

Info

Enables **temporal learning**

- Converts data into **time windows**
- Input → past 20 timesteps
- Output → next value

Neural ODE Architecture

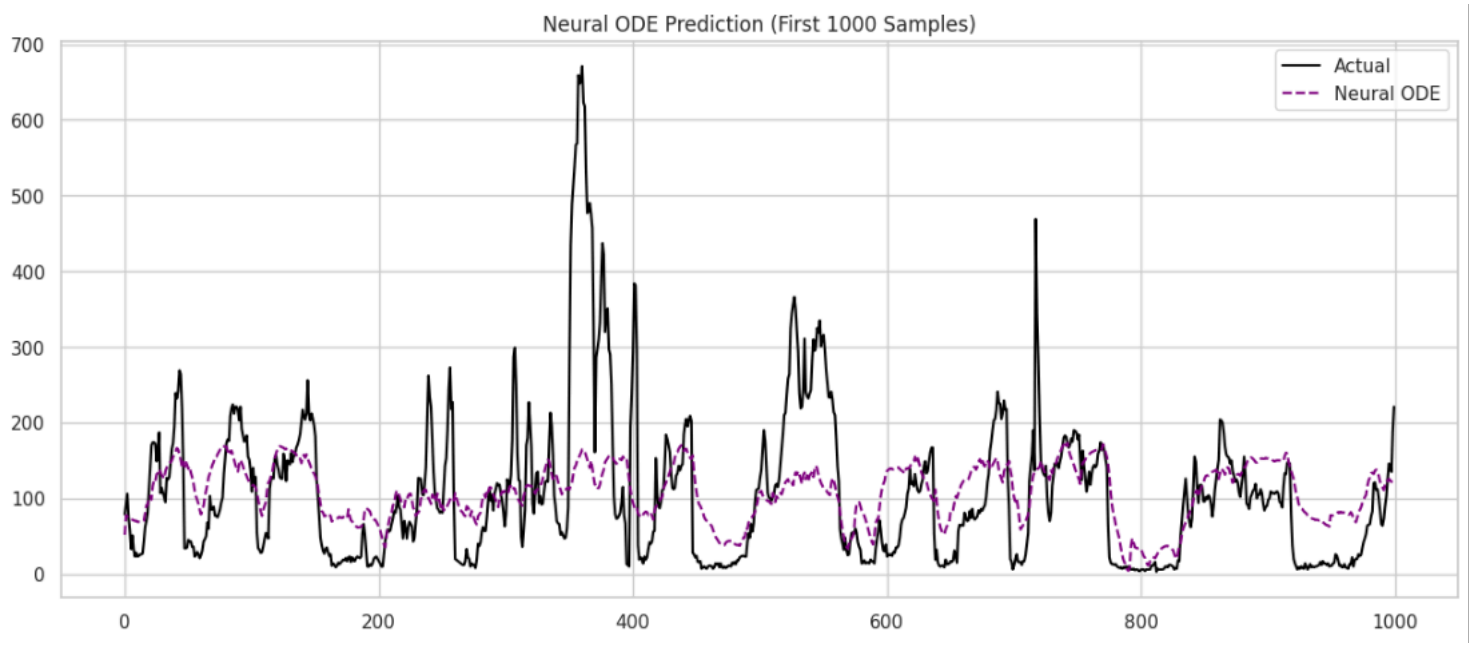


Key Insights

- GRU captures **temporal context**
- ODE models **continuous evolution**
- Final layer predicts pollution value

✓ **Results**

- MSE: 12859
- MAE: 97.66
- R^2: -0.46



⚠ **Warning**

- Poor performance → dataset is **not true ODE system**
- Hard for model to generalize

Use of XAI

✎ **Note**

- Traditional ML → interpretable (coefficients, feature importance)
- Neural ODE → **black-box model**

XAI helps us *understand model behavior .

So this is how XAI can be used with Neural ODE Model in general case

---

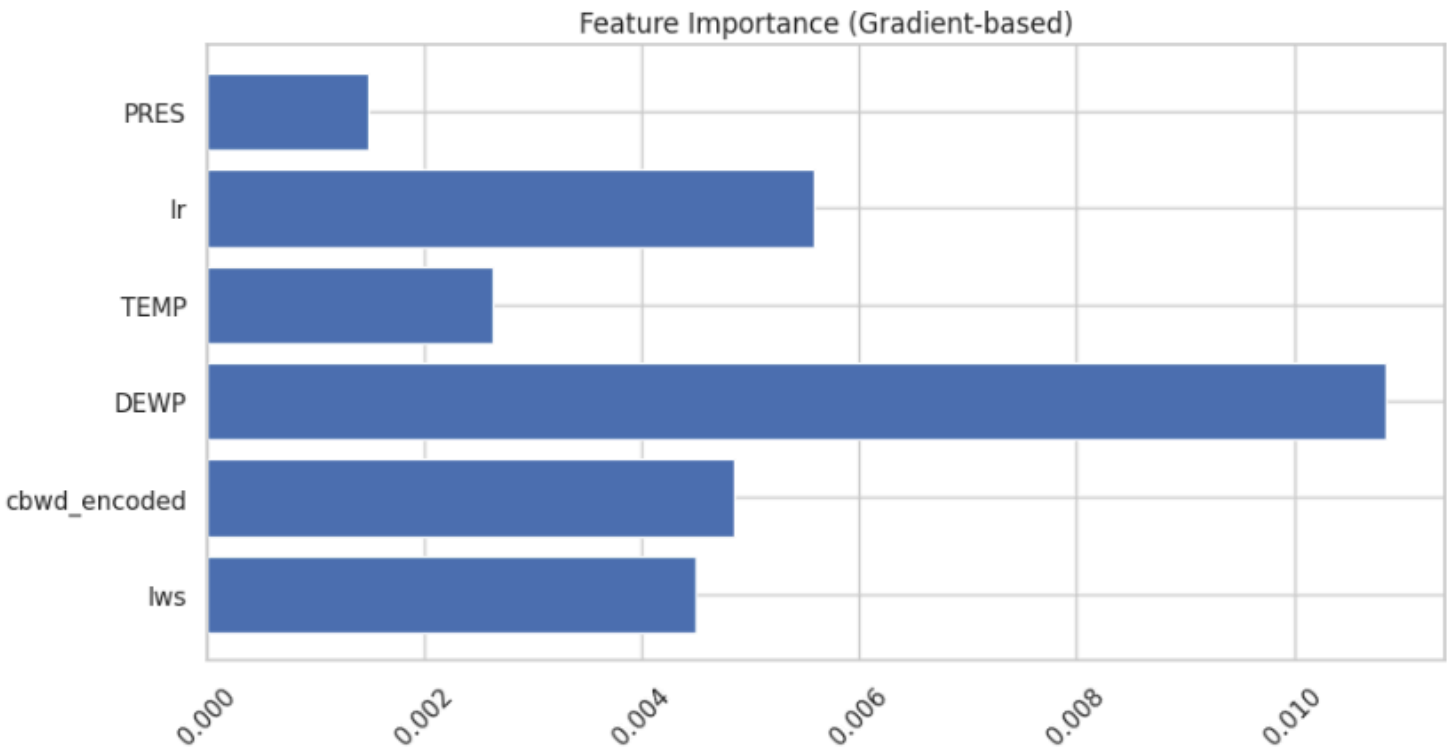
Gradient-Based XAI

```
output.sum().backward()
grads = X_sample.grad.abs()
```

Info

- Computes **gradient of output w.r.t input**
- Measures **sensitivity**

Feature Importance



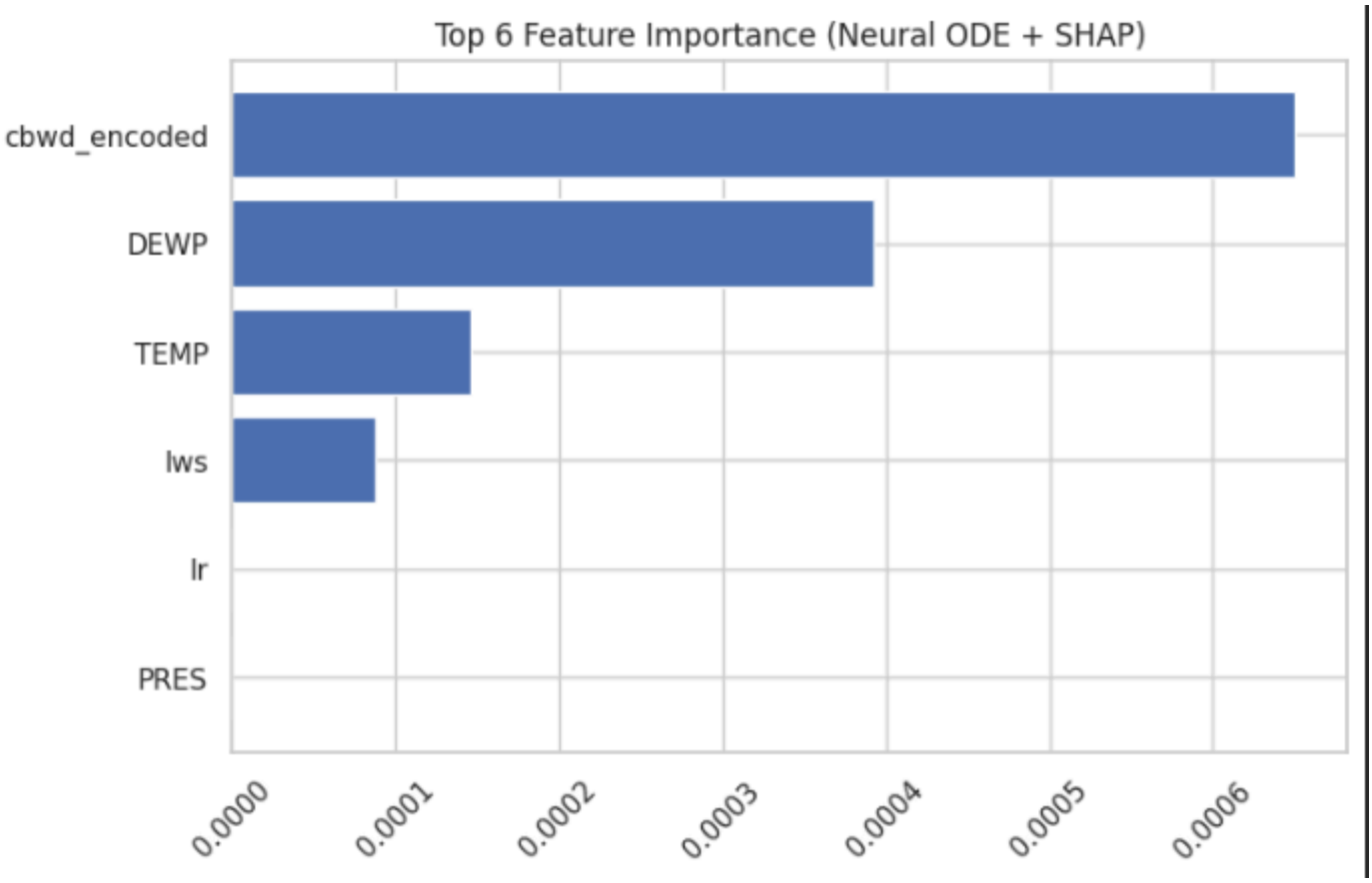
Observation

- **DEWP** most important
- Followed by **Ir, cbwd, Iws**

SHAP (Kernel SHAP)

Info

- Estimates **feature contribution using game theory**
- Works for any model



Observation

- **cbwd_encoded** most important
- Followed by **DEWP, TEMP**

Conclusion

XAI Insights

- Helps explain:
  - Feature importance
  - Model behavior
- Different models highlight **different important features**

Key Takeaway

- Neural ODE → best for **true dynamical systems**
- XAI → best for **interpretability**

Combining both:

- Useful for **real-world understanding**
- But Neural ODE + XAI is still at an learning process

Code Reference

See implementation here: `node_xai_beijing.ipynb`